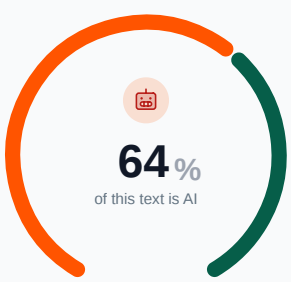
AI Detection Report for test (4).pdf

Oct 5, 2026 | 5,423 Words | Pangram 4.0

Summary

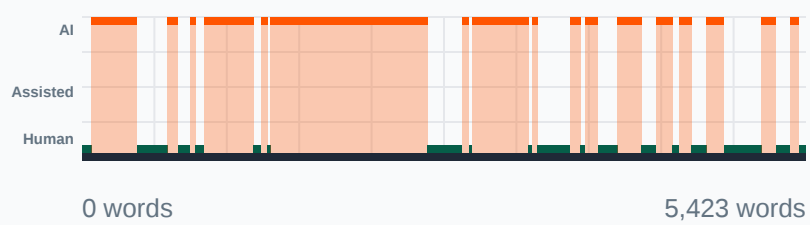
We believe that this text is a mix of AI and human-written content.

Authorship Breakdown



64 % of the document	AI Generated
0 % of the document	AI Assisted
36 % of the document	Human Written

Authorship Progression



Analyzed Text

Words that are underlined indicate AI phrases, while the numbers represent how frequently they are likely to appear in AI writing. These elements are not used to determine the results but are displayed separately as patterns commonly found in AI-generated text.

Human Written | 63 Words | High Confidence

Predictive KV Cache Amortization for Edge Language Model Inference on Mobile Devices Arjit Tripathi arjit.tripathi2023@vitstudent.ac.in Heer Shah shah.heer2023@vitstudent.ac.in Medha Sriram medha.sriram2023@vitstudent.ac.in Dr. Shalini L l.shalini@vit.ac.in Server-based systems sidestep these repeated prefills us- ing a shared key-value (KV) store to maintain state across multiple user requests.

AI | 338 Words | High Confidence

SGLang keeps prompt prefixes in a radix tree [6], vLLM shares KV blocks between requests [7], Prompt Cache stores attention states for reusable prompt modules [8], and RAGCache caches the KV state of retrieved documents [9]. However, this approach relies heavily upon there being some number of users requesting similar queries. Since there is not a stream of multiple requests coming from a single phone, server-based solutions do not apply. The request that would populate the cache is the user’s own first turn in a new setting, and that turn pays the full prefill cost. Abstract—On-device language models often need situational context prepended to the user’s input: location, nearby people, time of day. The prefill step that reads this context is expensive. On a OnePlus

AI | 338 Words | High Confidence

11R running Qwen2.5-0.5B-Instruct (Q4_K_M), prefilling a 439-token context prompt took 14.2 s. Server systems cache KV state across users to skip repeated prefill, but that depends on overlapping requests from multiple users, which a single phone does not have. A phone does have sensors. CAP- KVC uses GPS, BLE proximity, and time of day to rank a small set of pre-written contexts, prefills the likely ones during idle time, and keeps up to three as resident llama.cpp KV state. At interaction time only the user's intent tokens are decoded. With a cached 44-token context, the CAP-KVC prefill step (9 intent tokens) averaged 351 ms versus 2507 ms for the matched 52-token inline condition, a 7.1× reduction (n = 30 each); the KV state restore added 2.4 ms. Over 100 synthetic days with five contexts, correct-context selection reached 98.5% of 6,400 interactions versus 92.2% for request-driven LRU caching, at a cost of 1.5% wrong-context selections. Undetected GPS forgery raised wrong-context selections to 38.9%. One device, one model, five contexts, synthetic traces. Index Terms—Key-value cache, edge inference, predictive caching, language models, mobile systems, sensor-driven context.

Human Written | 247 Words | High Confidence

I. INTRODUCTION While there is no stream of requests coming from a phone, the device does have sensors available to help inform which context the user is likely going to want. In this paper we pro- pose CAP-KVC (Context-Aware Predictive KV-Cache prim- ing). It makes predictions on the best few context descriptions based upon the user's current location (via GPS), who or what else is nearby (via Bluetooth Low Energy, or BLE, proximity), and what time of day it is. While the phone is in idle mode it prefills those predicted contexts the user is most likely to want next. Once prefilled, the phone stores up to three of them as llama.cpp KV state. When the user decides to interact with the device, only their intent tokens need to be decoded. On a OnePlus 11R running Qwen2.5-0.5B-Instruct (Q4_K_M), we measured that restoring an existing 44-token context as opposed to prefilling it—parsing all of its tokens inline— takes 351 ms vs. 2507 ms respectively (7.1×, n = 30 each). Restoring the actual state takes 2.4 ms. In order for a language model to respond effectively to a question or task posed by a user from their smartphone, it needs to know where the user is physically located as well as other factors about the user's environment—for instance, who else may be in the user's vicinity and what time of day it currently is.

AI | 80 Words | High Confidence

Take an AAC (augmentative and alternative communication) app [1]: the user taps "I need water," and the app has to expand that into a full sentence. But what it should say depends on the setting. A hospital ward at 11 a.m. calls for different phrasing than home in the evening with a caregiver present, so the context has to be there. The standard approach is to prepend a context description to the intent and let the model read both.

Human Written | 85 Words | High Confidence

Our study was performed over a synthetic period of 100 days of sensor data for five different contexts. The routing and residency policies were able to determine the correct context for 98.5% of 6,400 interactions generated during that time frame, compared with 92.2% for a request-driven cache with no ability to leverage sensing capabilities, which misses on every first visit. The evaluation is limited to a single device, a single model, five contexts, and synthetic traces. This paper makes three contributions.

AI | 42 Words | High Confidence

First, a sensor-driven predictive KV caching design for a single user on a phone: deterministic context scoring from GPS, BLE and time-of-day readings, a bounded resident set with hysteresis-gated replacement, and a two-tier fallback for cold misses.

Human Written | 66 Words | High Confidence

Second, an Android implementation built on llama.cpp through JNI, However, feeding this information into the language model prior to submitting a query is very costly. We found that on an Android-based smartphone, populating a 439-token input prompt took 14.2 s (Section V-B). Even a short 52-token prompt took 2.5 s, which works out to roughly 30 ms per token.

AI | 358 Words | High Confidence

For someone who communicates through the device, that kind of delay comes up every single turn.FFTABLE I WHERE CAP-KVC DIFFERS FROM REQUEST-DRIVEN KV REUSE. before the first output token. That is the gap CAP-KVC is meant to address. Request-driven reuse CAP-KVC III. SYSTEM DESIGN GPS, BLE, time Setting Multi-user server One user, one phone Cache filled When a request arrives Before it, while idle Selection signal Prompt prefix / docu- ment Full prefill Restore if predicted First request in a context The architecture has two independent paths (Fig. 1). A background sweep runs every 60 s, reads sensors, scores each context, and updates which KV states are resident. The interaction path is triggered by user input: it picks the best resident state and decodes the intent. Failure mode Cold miss Wrong context or cold miss A. Problem Let C = {c1, . . . , cM} be pre-written contexts with prompts pi of Ni tokens. In the straightforward approach (the retrieval- augmented style), the device decodes the full pi followed by the user's intent q on every request; prefill cost grows with Ni + |q|. CAP-KVC moves the prefill of pi to idle time and stores the resulting KV state. At interaction time the device restores that state and decodes only q. with per-slot KV residency, rollback of each interaction so a resident state can be reused unchanged, crash-safe cache persistence via atomic file renames, and location input vali- dation. Third, an evaluation that separates the restore bene- fit (matched-context latency) from prediction quality (trace- driven comparison

AI | 358 Words | High Confidence

against oracle, request-driven, random and single-sensor baselines) and from adversarial sensor input (forged GPS and cloned BLE beacons). II. RELATED WORK The question is which contexts to prepare, since memory allows only K < M of them to be resident at once. Our prototype has M = 5 contexts (home morning, home evening, hospital ward, therapy room, caregiver visit) and K = 3 resident slots. B. Context Scoring Each context ci is associated with an expected hour of day ti, an anchor GPS location ℓi, and a set of BLE beacon anchors Ai with weights wij.

Human Written | 63 Words | Medium Confidence

When a sensor reading comes in, the system computes a score for each context: S(ci) = αStime + βSgps + γSble, α + β + γ = 1, (1) where each component captures one kind of sensor evidence: (, σ = 2 h, (2) −dcirc(t, ti)2/2σ2) Stime = exp ( ) , λ = 0.1 km, (3) −dhav(ℓ, ℓi)/λ Sgps = exp /∑ j∈Ai wij ⊮[j seen] j∈Ai wij.

AI | 56 Words | High Confidence

(4) Sble = ∑ dcirc wraps around midnight so that 11:55 p.m. and 12:05 a.m. are treated as ten minutes apart rather than nearly 24 hours. Sgps uses the Haversine distance and falls to 0.37 at 100 m. The BLE score is the weighted fraction of expected beacons that the device actually detected.

Human Written | 25 Words | High Confidence

Reusing KV state across requests. Caching KV state to skip redundant prefill is well-studied on the server side [6], [7], [8], [10], [9], [11].

AI | 1153 Words | High Confidence

The approaches differ in granularity (prefix trees, shared blocks, precomputed modules, non-prefix chunks, per-document state, compressed transferable state), but they share a common assumption: the cache gets populated when a request containing the relevant content arrives, and the payoff comes when later requests overlap with it. RAGCache [9] is the most relevant to our scenario because it caches KV state for specific retrieved documents (not arbitrary prefixes) and uses a retrieval-aware eviction policy that outperforms plain LRU. Still, even RAGCache is reactive. On a multi- user server, request overlap is almost guaranteed. On a phone used by one person, the first request in a new context is the one that would populate the cache, and that is exactly the request where the user pays the full prefill cost. CAP-KVC populates the cache before any request, from sensor

AI | 1153 Words | High Confidence

evidence (Table I). We do not run SGLang or RAGCache; they require GPU servers. Section V-C simulates their request-driven policy (LRU over contexts identified at request time) on the same synthetic traces. Shrinking the KV cache. H2O [12] and Scissorhands [13] evict less important tokens; SnapKV [14] compresses keys and values; StreamingLLM [15] keeps only attention sinks and recent tokens; KVQuant [16] quantizes the stored state. These reduce memory within a sequence. They are orthogonal to what we do—a compressed KV state can still be primed and restored. The weights α, β, γ are not learned—they come from a per- sensor reliability heuristic. Time gets a fixed wt = 0.4. GPS gets wg = exp(−a/50 m) for reported accuracy a (0 without a fix). BLE gets wb equal to the mean of 1/(1+e−0.1(r+70)) over the observed RSSI values r (0 when no beacons are detected). These are normalized to sum to one. Contexts scoring below 0.01 are dropped. “Predictive” in the title refers to preparing a context before the request, not to a trained model. C. Residency and Hysteresis On-device inference. Quantization [4], [5], small-model design [17], runtimes like llama.cpp [2] and MLC-LLM [18], flash offloading [19], and heterogeneous NPU/CPU execu- tion [20] have collectively made it feasible to run language models on a phone. They lower the cost of processing each token, but they do not get rid of the context prefill that happens At most K = 3 KV states can be resident. We picked three as a design parameter: with five possible contexts, keeping three resident covers the majority of transitions while limiting how much background priming the system has to do.FFBackground: sweep every 60 s; BLE-free check every 15 min ranking load Sensor sweep GPS fix (validated) Context scoring αStime + βSgps + γSble BLE scan, 3 s Priming (idle) prefill context into slot, save KV, atomic rename ranks 5 contexts Residency policy at most 3 resident; replace only if score gain exceeds Δ= 0.10 clock save llama.cpp context, 4 sequences KV files (excluded from backup) slot 0 resident slot 1 resident slot 2 resident slot 3 scratch restore on load resident KV Interaction hit Select highest-ranked resident Resume slot decode intent only, Intent buttons or gaze dwell Reply text, TTS context (latest sweep) Tier 1 canned reply shown at once roll slot back cold miss →Tier 2: start priming current top contexts Fig. 1. CAP-KVC architecture. A background sweep scores five pre-written contexts from GPS, BLE and time, keeps up to three of them as primed KV state in llama.cpp sequence slots, and saves each state to disk. An interaction shows a canned reply, then resumes the highest-ranked resident context and decodes only the intent; if none is resident, priming starts in the background. Slot 3 serves inline inference without disturbing resident states. Native boundary. 14 JNI functions, all scalar arguments and results: lifecycle (initialize backend, load model, release), KV operations (save, load, clear scratch, reset slot), inference (inline run, prefill only, resume), and four timing getters. A sin- gle ReentrantLock serializes every native call; llama.cpp context state is shared and not thread-safe. After each sweep the scorer ranks all contexts. Candidates already resident stay. A new candidate takes a free slot if one exists; if all three are occupied, it replaces the weakest resident only when its score exceeds the weakest’s by more than Δ. Why the margin? Without it, noisy GPS accuracy and intermittent BLE detection caused the system to keep swapping states in and out between sweeps. Δis a threshold on scores, unrelated to K. An earlier draft conflated the two and justified Δwith an unrelated text-similarity experiment; we drop that justification. The deployed value is Δ= 0.10; Section V-D measures its effect. Sequence slots. We set up the llama.cpp context with four sequences of 1024 tokens each. Three of these are reserved for resident states, and the fourth serves as a scratch space for inline inference and benchmark runs. We added the scratch sequence specifically so that running a benchmark or doing inline inference would not disturb a resident state that was already loaded. The same replacement rule runs in the 60-second sweep and in a 15-minute background check that omits BLE scanning and may only add

AI | 1153 Words | High Confidence

the top-ranked context. D. Interaction and Fallback Priming decodes the context into its assigned slot and serializes the slot to a temporary file, then renames atomically (no partial cache file on crash). Resuming appends the intent at the slot's current position, generates the response, then strips everything past the cached boundary—rolling the slot back so the resident state is reused unchanged next time. With 24 layers, 2 KV heads of dimension 64, and fp16 storage, Qwen2.5-0.5B needs 12 KiB of KV data per token. The four sequences reserve 48 MiB at context creation. We did not re-measure process memory after this layout change. When the user interacts, the app picks the highest-ranked resident context from the latest sweep. If one exists, the device resumes that state and decodes only the intent. If no ranked context is resident (a cold miss), a canned response goes out immediately (Tier 1) and priming of the top-ranked contexts starts in the background (Tier 2). A wrong-context answer is the other failure mode: a resident context is used but does not match the user's actual situation. IV. IMPLEMENTATION Sensing. A foreground service runs the sensor sweep every 60 s. Each sweep performs a 3-second BLE scan and requests a location fix. Every location provider's last fix is checked before the system trusts it: the fix must not be flagged as mock by the Android OS, it must be at most 120 s old, and the reported accuracy must fall between 2 and 500 m. If nothing passes, one fresh fix is requested (network provider preferred over GPS for speed).

Human Written | 253 Words | High Confidence

A WorkManager job runs every 15 minutes AACBridge is an Android app (Kotlin, API 26+, arm64-v8a) that uses llama.cpp [2] through JNI. The model is Qwen2.5-0.5B-Instruct [3] quantized to Q4_K_M; we chose it as the smallest instruction-tuned model available in llama.cpp's GGUF format that fits within the phone's memory constraints.TABLE II NATIVELY TIMED LATENCY COMPONENTS (MS), MEDIAN [IQR], n = 30 EACH. CAP-KVC RESTORES A 44-TOKEN CONTEXT AND DECODES THE 9 INTENT TOKENS. Condition (tokens) Prefill Generation Restore No context (9) 377 [288, 503] 5775 [2823, 6213] – Inline, matched (52) 2449 [2231, 2777] 7964 [7350, 9374] – Inline (89) 4116 [3677, 4487] 8589 [7822, 11294] – Inline (206) 7075 [5148, 8700] 6991 [4107, 10275] – Inline (439) 14024 [13219, 14941] 5912 [5467, 7003] – CAP-KVC (9) 331 [276, 355] 4395 [3985, 5110] 1.2 [1.0, 3.3] prefill_scaling.pdf without BLE. Intent input. On-screen buttons or gaze-dwell interface. An sEMG+gaze fusion classifier from earlier work is included as a prototype input but not evaluated here. Backup. App backup disabled. KV cache directory ex- cluded from cloud backup and device-to-device transfer. V. EVALUATION A. Setup Fig. 2. Prefill latency against tokens decoded (log scale). Points are trials; markers show median and IQR (n = 30 per condition). All latency measurements were taken on a OnePlus 11R (Snapdragon 8+ Gen 1, 8 GB RAM, Android 14), with 4 threads, batch size 512, greedy decoding, and a cap of 64 output tokens.

AI | 49 Words | High Confidence

Each condition ran 2 warm-up trials fol- lowed by 30 measured trials. Prefill and generation times come from std::chrono timestamps placed around the llama_decode call in native code;

AI | 49 Words | High Confidence

restore time is measured around the load call. All 180 trials and the analysis script are in the repository.

Human Written | 22 Words | High Confidence

after the context was restored) averaged 351 ms (SD 97). The matched inline prefill (all 52 tokens) averaged 2507 ms (SD 522).

AI | 433 Words | High Confidence

The ratio between the two prefill measurements is 7.1× (Welch t(31.0) = 22.2, p < 10−19; Mann–Whitney p < 10−10). A rough time to first token (restore + prefill + one generation step) is about 0.43 s for CAP-KVC, about 2.6 s inline. Inline prefill grew at about 30 ms per prompt token (OLS, R2 = 0.91; Fig. 2), reaching 14.2 s at 439 tokens where the CAP-KVC prefill was 40.5× shorter. That larger ratio reflects the cost CAP-KVC avoids when contexts happen to be long, not a matched speedup. We only measured restore for the 44- token cache and cannot say how it scales with context length. One problem with this particular run: the phone's gaze interface remained active during the benchmark and issued 45 interaction inferences between trials. These held the engine lock, and the trial timer started before the lock was taken, so the end-to-end totals of some trials include time the system spent waiting for the lock rather than doing inference. We therefore report only the natively timed components (prefill, generation, restore), which the lock wait does not affect. We do not report end-to-end totals. The app has since been updated to block interaction inference during benchmarks and to start the trial timer after acquiring the lock. B. Matched-Context Latency The question here is simple: how much prefill work can be removed when a previously prepared context is restored instead of decoded from scratch? The prefill saving should be kept in perspective. Generation dominated the full response: 4.4–8.6 s at median with 64 output tokens. So the 2.1 s saved in matched prefill shortens the overall response by much less than 7.1×; the benefit is concentrated in time to first token. Generation times varied across conditions and trials (IQRs above 1 s). We recorded no thermal or CPU-frequency data and cannot explain this variation. C. Prediction and Cache Policy The CAP-KVC condition holds a 44-token context in its cache. The matched inline condition decodes the same context plus the intent as a single 52-token prompt. We also ran larger inline conditions (89, 206, and 439 tokens) to see how inline prefill scales; the ratios between these and CAP-KVC grow by construction since the inline prompts are longer, and they are not matched comparisons. Table II has the results. Three quantities matter and should be kept distinct: the KV restore overhead (loading the cached state from disk) averaged 2.4 ms (median 1.2 ms).

Human Written | 29 Words | High Confidence

The CAP- KVC prefill component (decoding only the 9 intent tokens, Traces. We do not have recorded sensor traces from real users, so we generated synthetic ones (seed 42).

AI | 42 Words | High Confidence

Each of 100 simulated days follows the five-context routine 06:00–22:00, with each context boundary shifted per day by U(±0.5 h). A sweep runs every simulated minute; an interaction fires every 15 minutes (6,400 interactions total).

Human Written | 298 Words | High Confidence

Per-sweep noise parameters: GPS displacement N(0, 50 m), GPS dropout 10%, reported accuracy N(15, 10) m, per-beacon detection 80%TABLE III INTERACTION OUTCOMES OVER 6,400 SYNTHETIC INTERACTIONS (%) AND STATES LOADED PER DAY. Δ= 0.10 UNLESS NOTED. Policy / scenario Correct Wrong Cold Loads/d hysteresis_churn.pdf Oracle (always resident) 100.0 0.0 0.0 5.0 Request-driven LRU, no sensing 92.2 0.0 7.8 5.0 Random ranking 20.7 79.3 0.0 1011 Time only 92.2 7.8 0.0 5.0 GPS only 87.9 12.1 0.0 45.7 BLE only 92.6 6.9 0.5 115.8 All sensors (deployed) 98.5 1.5 0.0 31.6 All sensors, Δ= 0 98.5 1.5 0.0 65.9 GPS 70% unavailable 98.2 1.8 0.0 116.7 High noise 90.6 9.4 0.0 106.2 GPS forged, not detected 61.1 38.9 0.0 18.2 GPS forged, mock flag rejected 98.3 1.7 0.0 75.8 Cloned BLE beacons 90.1 9.9 0.0 103.2 with RSSI N(−65, 10) dBm, BLE outage 15%, spurious beacon 10%. The high-noise variant: ±1.5 h boundary shifts, 200 m GPS displacement, 30% GPS dropout, 50 m accuracy, 50% detection, 40% BLE outage. One thing we should be upfront about: we wrote the context time anchors from the same daily routine that the traces follow, so time-of-day evidence is favourable to our system by construction. Fig. 3. States loaded per day against hysteresis margin Δ(100 synthetic days each). The share of correct interactions stayed at 98.5% (normal) and 90.6% (high noise) across all margins up to 0.5. Simulation.

AI | 78 Words | High Confidence

The trace evaluator runs the production scoring, residency and selection code (the same code deployed on the phone). It does not model priming delay: a state chosen at one sweep is assumed usable by the next simulated minute. in 0.5% of interactions (no registered beacon detected). Time alone reached 92.2%, but that number reflects how the traces were built—the time anchors match the daily schedule—not a property we would expect from real user schedules.

Human Written | 36 Words | High Confidence

Metrics. For each interaction, let c∗be the true context and ˆc be the one the system actually used. Correct: ˆc = c∗. Wrong context: ˆc̸ = c∗(the system picked a resident context but the wrong one).

AI | 90 Words | High Confidence

Cold miss: no ranked context is resident. These partition every interaction into exactly one outcome. Loads per day counts how many times a state was primed or restored into the resident set—a measure of background work. Cold misses were near zero for every sensing policy. The router's top-1 accuracy matched the share of correct interac- tions in every configuration. With three of five contexts always resident, the dominant failure mode is a wrong selection rather than a miss, so Hit@3-style metrics carry little information here.

Human Written | 155 Words | High Confidence

Random ranking also had no cold misses (three of five are always resident) but was correct only 20.7% of the time. D. Hysteresis Baselines. Oracle: c∗always resident (upper bound). Request-driven LRU: no sensing; the correct context becomes known only at request time, is used if cached, otherwise is a cold miss and enters a three-entry LRU cache persisting across days. Random: uniform random scores per sweep, same residency rule. Single sensor: time only; GPS only or BLE only with wt = 0.001 so time does not leak through normalization. With all sensors, the deployed rules picked the correct context 98.5% of the time (Table III). Fig. 3 shows what happened when we varied Δwith everything else held fixed. Raising Δfrom 0 to the deployed 0.10 halved loads per day under normal noise (65.9 to 31.6) and cut them from 297.5 to 106.2 under high noise.

AI | 186 Words | High Confidence

Accuracy did not change at any margin up to 0.5—consistent with the interpretation that genuine context changes move the scores by much more than these margins. Each load is a disk restore or (if the cache file is missing) a full prefill, so the margin mainly saves background work. A margin of 0.3 would reduce loads further on these traces at no accuracy cost, but we keep 0.10 until real traces are available; a larger margin also delays switching when sensor evidence shifts by a small but genuine amount. E. Not Measured The comparison with request-driven LRU deserves some unpacking. That baseline scored 92.2% and never once se- lected a wrong context, but that is by definition—it only uses a context it already knows is correct. Its 7.8% failure rate is entirely cold misses. The reason is structural: five contexts visited in order with only three cache entries means every first visit to a new context after the cache fills is a miss. Sensing eliminated those first-visit misses, but traded them for 1.5% wrong-context selections.

Human Written | 103 Words | High Confidence

GPS alone could not separate the three contexts at the home location. BLE alone left the system with no candidate Battery drain, power, temperature, CPU frequency: un- measured for the 60-second sweep, the 15-minute job, and background priming. Each load in Table III is a unit of background work whose energy cost we have not been able to quantify. 1.5% wrong-context selections. The main vulnerability we identified is undetected GPS forgery, which raised wrong- context answers to 38.9%. VI. THREAT MODEL Code, benchmark logs, evaluation harness, and figure scripts are available at https://github.com/omni-ar/AACBridge.

AI | 117 Words | High Confidence

ACKNOWLEDGMENT Two assets matter: the context selection (it determines what the device says for the user) and the cached KV files (they encode the full context prompts). We consider three attackers. (A) An app on the same phone supplying mock locations via Android's API. (B) Someone nearby broadcasting BLE beacons that clone registered anchor identifiers. (C) Someone with file-level access to the phone or backups. Scope of attack. The router only ranks the five pre-written contexts. Sensor manipulation can force a wrong selection but cannot add a context, change a prompt, or alter the user's intent. A wrong context still changes what the model produces, so the damage is bounded but not eliminated.

Human Written | 49 Words | High Confidence

The authors used Claude (Anthropic) to draft the text of this revised manuscript (all sections), which the authors then edited; to help write and revise code in the Android application, the JNI layer, the trace evaluator and the analysis and plotting scripts used for Tables II–III and Figs.

AI | 84 Words | High Confidence

2–3; and for language editing. Google Antigravity was used as an AI-assisted development tool for parts of the Android/JNI implementation (Section IV). Fig. 1 is drawn with TikZ code, written with Claude's assistance from a component layout specified by the authors; Figs. 2–3 are rendered from measured or simulated data by scripts in the repository. No figure was produced by an image-generation model. Research questions, system design decisions, experiment choices and the interpretation of results are the authors'.

Human Written | 105 Words | High Confidence

REFERENCES [1] D. R. Beukelman and J. C. Light, Augmentative and Alternative Commu- nication: Supporting

Human Written | 105 Words | High Confidence

Children and Adults with Complex Communication Needs, 5th ed. Baltimore, MD, USA: Paul H. Brookes, 2020. [2] G. Gerganov et al., “llama.cpp: LLM inference in C/C++,” 2023. [Online]. Available: https://github.com/ggerganov/llama.cpp [3] A. Yang et al., “Qwen2.5 technical report,” arXiv:2412.15115, 2024. [4] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, “LLM.int8(): 8-bit matrix multiplication for transformers at scale,” in Proc. NeurIPS, 2022. [5] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, “GPTQ: Accurate Mitigations and residual risk.

AI | 128 Words | High Confidence

We reject location fixes flagged as mock by Android, along with stale fixes and those with reported accuracy outside 2–500 m. With the mock flag working, forged GPS had almost no effect (1.7% wrong context, Table III). On a rooted phone a forged fix need not carry the flag; if it also reports plausible accuracy (e.g. 5 m), our checks accept it, and the wrong-context rate reached 38.9%. BLE advertisements are unauthenticated—anyone can broadcast any beacon identifier—and we have no defence against cloned beacons (9.9% wrong context). The router does not cross-check GPS against BLE. Cache files are stored unencrypted in app-private storage, excluded from backup and device transfer; protection extends only as far as the Android sandbox.

Human Written | 269 Words | High Confidence

post-training quantization for generative pre-trained transformers,” in Proc. ICLR, 2023. [6] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, Out of scope. Rooted/compromised OS, kernel or firmware attackers, physical extraction of storage. VII. LIMITATIONS AND CONCLUSION C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng, “SGLang: Efficient execution of structured language model programs,” in Proc. NeurIPS, 2024. [7] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with PagedAttention,” in Proc. SOSP, 2023. [8] I. Gim, G. Chen, S.-S. Lee, N. Sarda, A. Khandelwal, and L. Zhong, “Prompt Cache: Modular attention reuse for low-latency inference,” in Proc. MLSys, 2024. [9] C. Jin, Z. Zhang, X. Jiang, F. Liu, S. Liu, X. Liu, and X. Jin, “RAG- Cache: Efficient knowledge caching for retrieval-augmented generation,” ACM Trans. Comput. Syst., vol. 44, no. 1, Art. 2, Feb. 2026, doi: 10.1145/3768628. [10] J. Yao et al., “CacheBlend: Fast large language model serving for RAG with cached knowledge fusion,” in Proc. EuroSys, 2025. [11] Y. Liu et al., “CacheGen: KV cache compression and streaming for fast large language model serving,” in Proc. ACM SIGCOMM, 2024. [12] Z. Zhang et al., “H2O: Heavy-hitter oracle for efficient generative inference of large language models,” in Proc. NeurIPS, 2023. [13] Z. Liu et al., “Scissorhands: Exploiting the persistence of importance There are several things we have not measured and several ways this evaluation is narrower than we would like.

AI | 105 Words | High Confidence

The latency numbers come from one phone, one 0.5B model, 30 trials per condition, one cached context size. We do not know how restore time scales with context length, what end-to-end latency looks like without the gaze-interface interference, how much memory the slot layout actually uses, or how our phone compares on an independent llama.cpp benchmark. The pre- diction numbers come from synthetic traces whose schedule matches the contexts' time anchors; the simulation does not model priming delay. Real multi-day sensor recordings from actual users are needed. Five hand-written contexts. Energy cost unmeasured. No evaluation with AAC users.

Human Written | 100 Words | High Confidence

No clinical claim. hypothesis for LLM KV cache compression at test time," in Proc. NeurIPS, 2023. [14] Y. Li et al., "SnapKV: LLM knows what you are looking for before generation," in Proc. NeurIPS, 2024. [15] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, "Efficient streaming language models with attention sinks," in Proc. ICLR, 2024. [16] C. Hooper et al., "KVQuant: Towards 10 million context length LLM inference with KV cache quantization," in Proc. NeurIPS, 2024. [17] Z. Liu et al., "MobileLLM: Optimizing sub-billion parameter language models for on-device use cases," in Proc. ICML, 2024.

AI | 67 Words | High Confidence

Within those limits, the measurements do separate the parts of the question. The KV restore overhead was 2.4 ms; the matched prefill dropped from 2507 ms to 351 ms. Sensor- driven residency was able to select the correct context for 98.5% of synthetic interactions, compared with 92.2% for request-driven LRU, by trading first-visit cold misses for [18] MLC Team, "MLC LLM," 2023.

Human Written | 49 Words | High Confidence

[Online]. Available: https://github.com/ mlc-ai/mlc-llm [19] K. Alizadeh et al., "LLM in a flash: Efficient large language model inference with limited memory," in Proc. ACL, 2024. [20] Z. Xue et al., "PowerInfer-2: Fast large language model inference on a smartphone," arXiv:2406.06282, 2024.